

Introduction to Algorithm

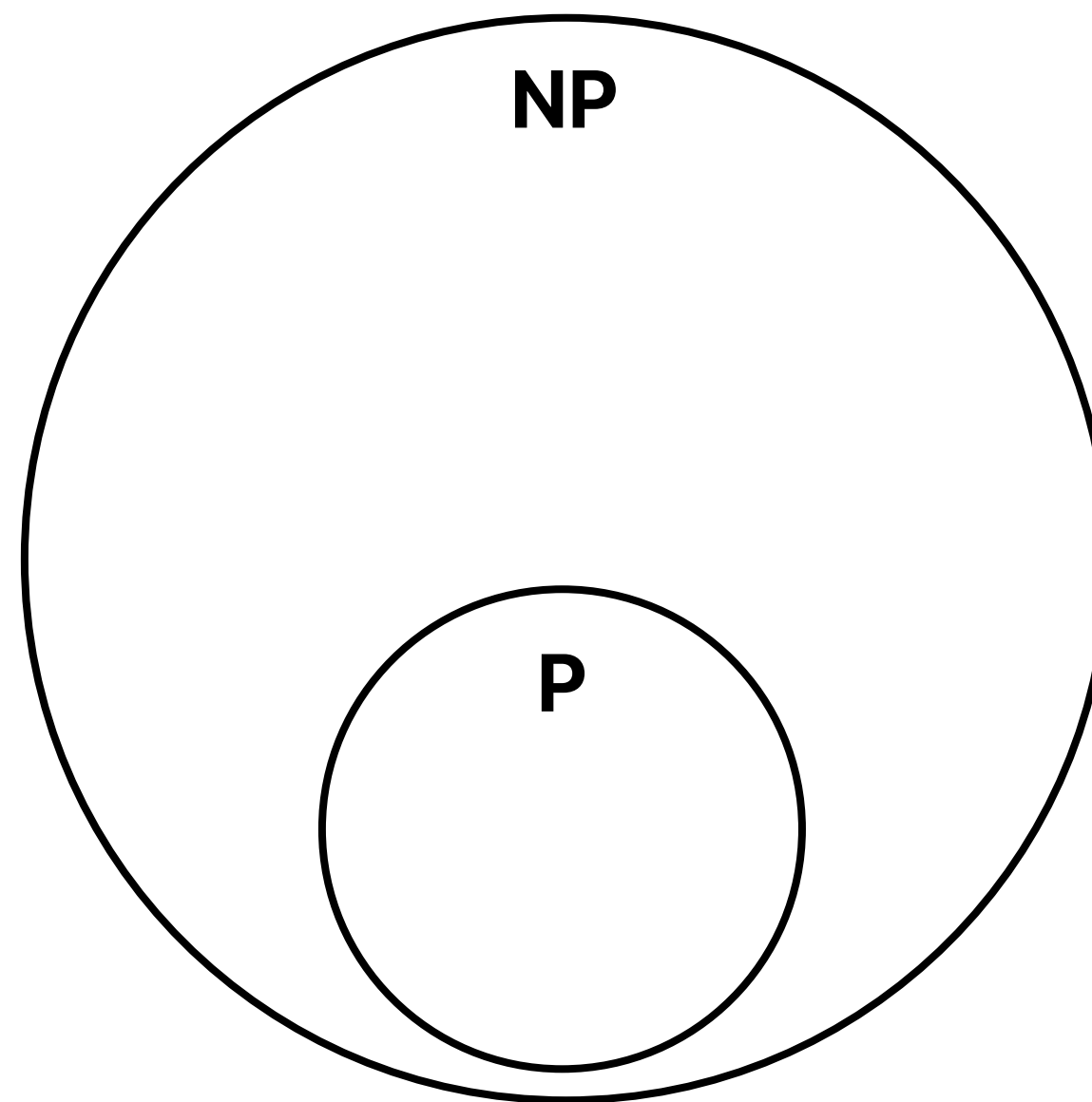
기초 알고리즘

Intractable 문제 사례 탐색

NP-Complete

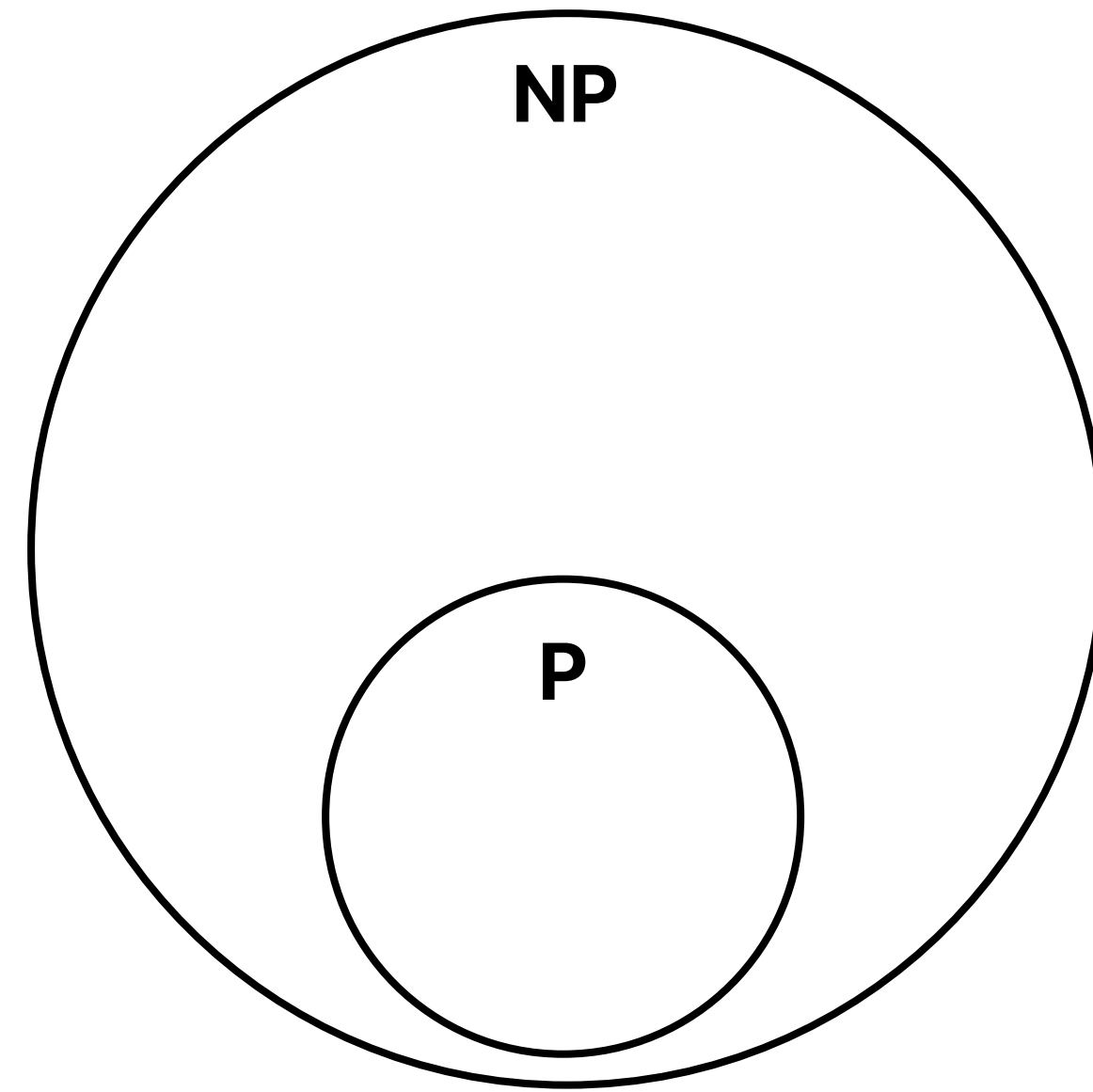
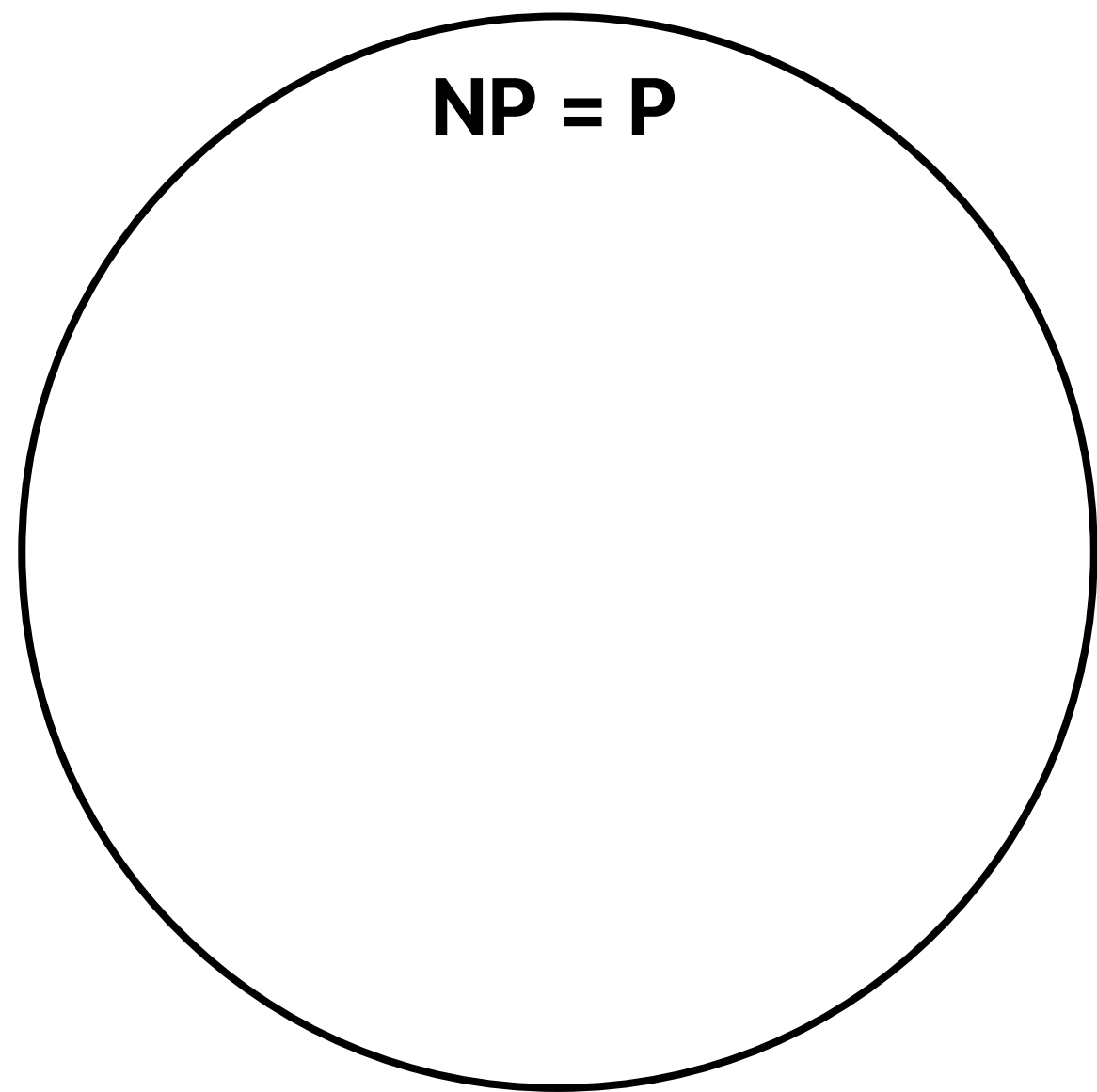
P vs. NP

- 모든 P 문제는 NP 문제에 포함된다



P vs. NP

- 모든 NP문제는 P문제에 포함된다($P = NP$)
- 또는 모든 NP문제가 P문제에 포함되는 건 아니다($P \neq NP$)



NP-Complete

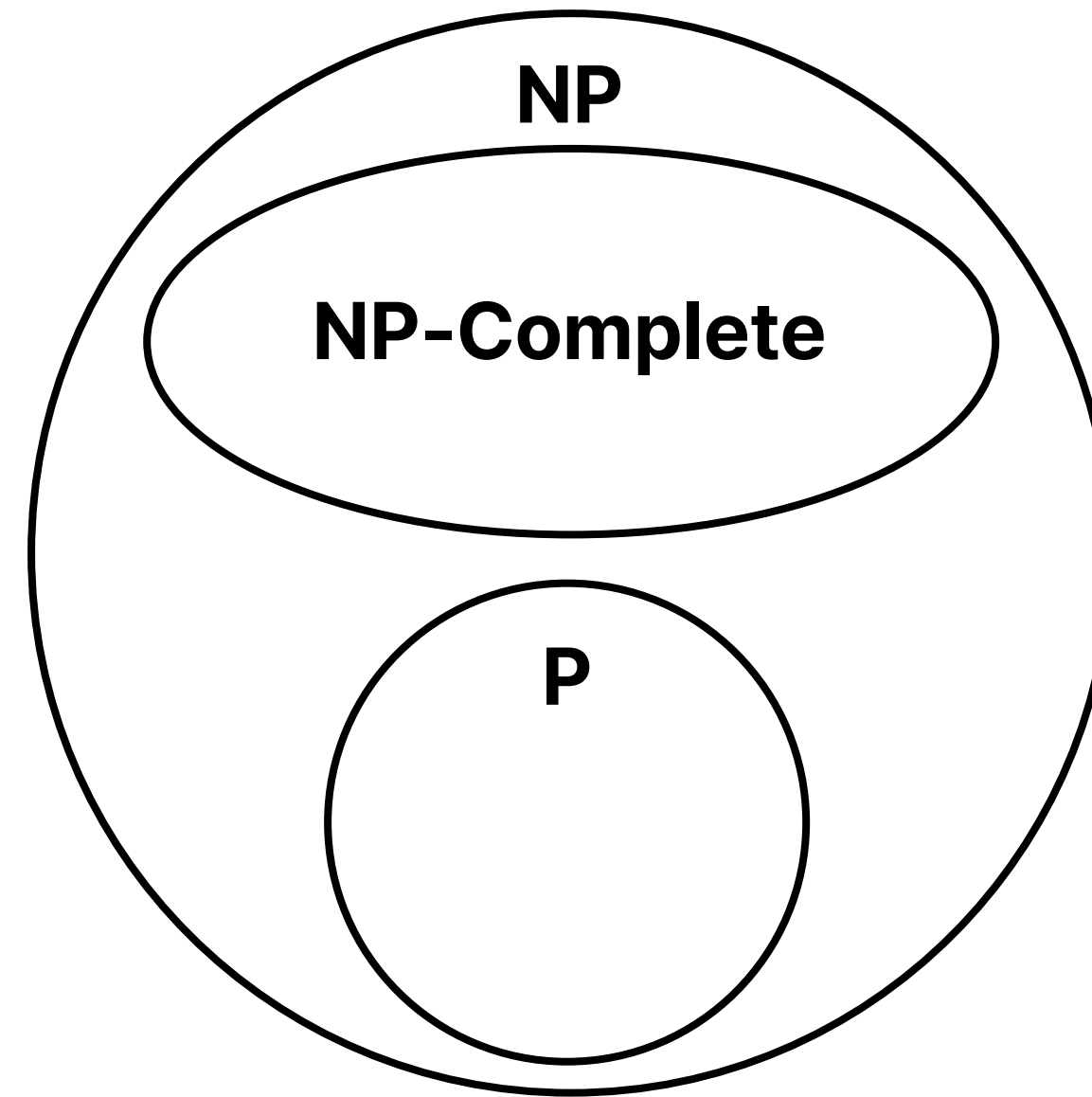
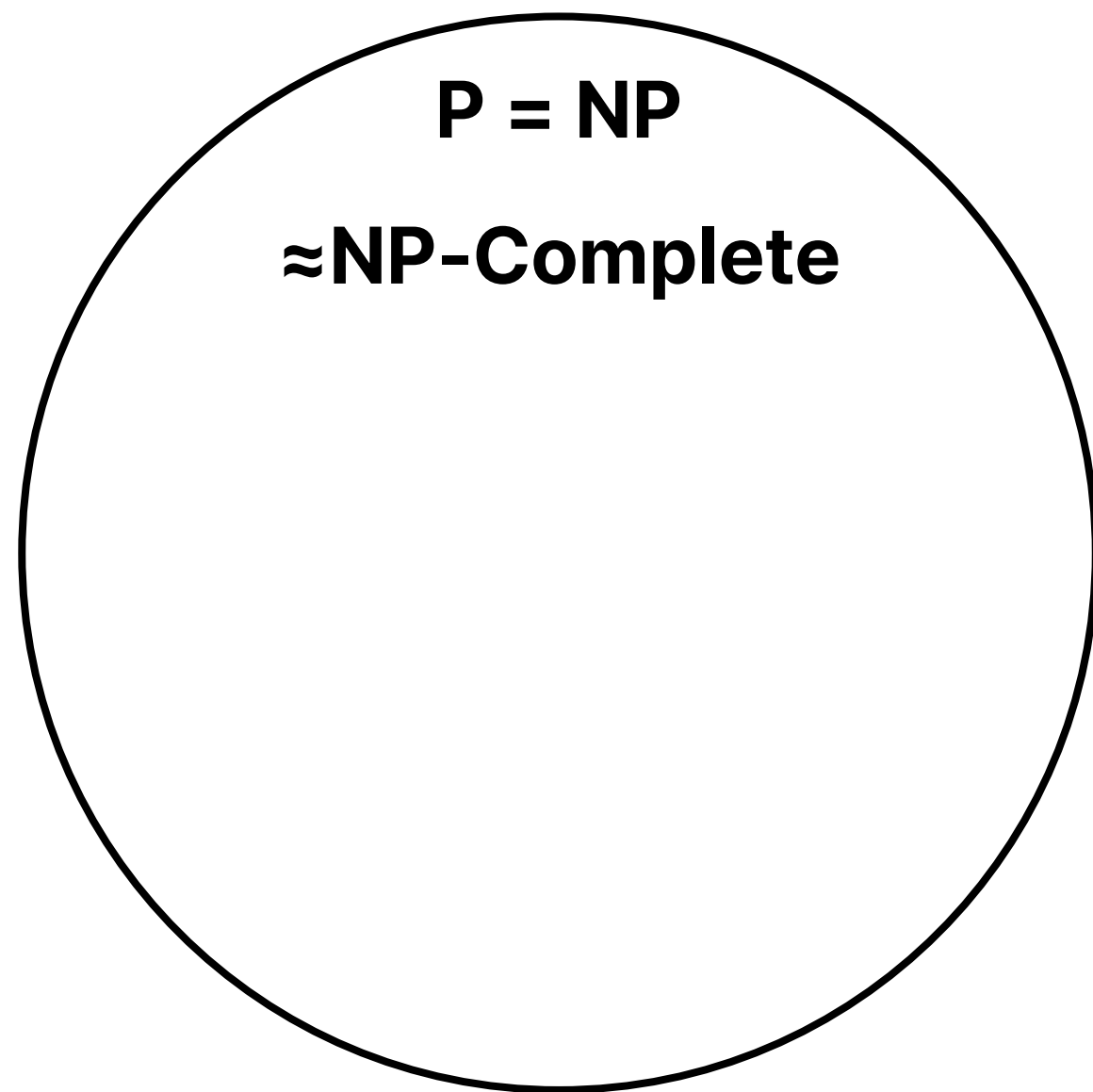
- NP 문제들 중에서도 "가장 어려운 핵심 문제들"
- NP-Complete는 모든 NP 문제를 자신으로 환원시킬 수 있는 강력한 속성을 가짐
- 환원(Reduction): 문제 A를 문제 B로 변형하여 풀 수 있다면, "A를 B로 환원한다"라고 표현. 만약 B를 쉽게 풀 수 있다면, B로 변형된 A도 쉽게 풀 수 있다는 것을 의미함.

NP-Complete

- 그러므로, NP-Complete 문제를 하나라도 다항 시간 내에 풀 수 있다면, 모든 NP 문제도 P 문제에 포함된다($P = NP$)
- 즉, NP-Complete는 NP의 운명을 결정짓는 '마스터 키'와 같은 존재!

NP-Complete

- NP-Complete 문제를 하나라도 다항 시간 내에 풀 수 있다면, $P = NP$
- 그렇지 않다면... $P \neq NP$





NP-Complete

- 실무에서 우리가 마주치는 난제들의 90% 이상은 이 NP-Complete 영역에 속함
- 대표적인 NP-Complete 문제
 - SAT (Satisfiability Problem)
 - TSP (Traveling Salesperson/Salesman Problem)
 - 0/1 Knapsack Problem (배낭 문제)

대표적인 NP-Complete 문제: SAT (Satisfiability Problem)



SAT (Satisfiability Problem)

- 여러 논리 변수들이 AND, OR로 얹힌 식을 참(True)으로 만드는 조합이 있는지 묻는 문제
- 역사상 최초로 NP-Complete임이 증명된 문제로, 모든 컴퓨터 논리의 기초가 됨
- 변수의 개수가 n 개일 때 조합은 2^n 으로 늘어나며, 현대 보안 시스템의 근간이 됨
- 이 문제를 빨리 푸는 알고리즘이 나오면, 현대의 암호 체계가 무너질 수 있음



SAT (Satisfiability Problem)

SAT (Satisfiability Problem) 예제

- 3개의 변수와 논리식이 주어집니다
 - 변수: A, B, C
 - 논리식: (A or not B) and (not A or C) and (B or not C)
- 각 변수는 0(False) 또는 1(True) 중 하나의 값을 가집니다
- 이 논리식을 참(True)으로 만드는 모든 조합을 찾아 내세요



SAT (Satisfiability Problem) 예제

- 변수는 0(False) 또는 1(True) 중 하나의 값을 가집니다
 - A: 0 or 1
 - B: 0 or 1
 - C: 0 or 1



SAT (Satisfiability Problem) 예제

- 가능한 모든 조합의 개수는
 - ➔ 변수가 3개라면, 2^3
 - ➔ 변수가 N개라면, 2^n

	A	B	C
1	0	0	0
2	0	0	1
3	0	1	0
4	0	1	1
5	1	0	0
6	1	0	1
7	1	1	0
8	1	1	1

대표적인 NP-Complete 문제: TSP (Traveling Salesperson Problem)

TSP (Traveling Salesperson Problem)

- 외판원이 여러 도시를 딱 한 번씩만 방문하고 출발지로 돌아오는 최단 경로를 찾는 문제
- 물류 배송, 회로 기판 설계, 유전자 서열 분석 등 수많은 분야에서 등장하는 난제
- 단순해 보이지만 도시가 늘어날수록 경로의 가짓수가 팩토리얼($n!$)로 폭증
- 전형적인 NP-Complete 문제로, 도시가 조금만 많아져도 문제를 푸는데 슈퍼컴퓨터로도 수만 년이 걸릴 수 있음

TSP (Traveling Salesperson Problem) 예제

- 도시간 거리 행렬(Adjacency Matrix)이 주어집니다
- 도시: 도시 1, 도시 2, 도시 3, 도시 4
- 도시간 이동 비용이 숫자로 주어집니다
- 가능한 모든 방문 순서를 생성하고, 총비용이 최소인 경로를 찾아내세요

	도시 1	도시 2	도시 3	도시 4
도시 1	0	10	15	20
도시 2	10	0	35	25
도시 3	15	35	0	30
도시 4	20	25	30	0

TSP (Traveling Salesperson Problem) 예제

- 시작 및 종료 도시를 '도시 1'로 고정했을 때, 가능한 경로의 수

➔ 나머지 3개 도시를 순서대로 선택하는 경우의 수: 6개

경로	1	2	3
1	도시2	도시3	도시4
2	도시2	도시4	도시3
3	도시3	도시2	도시4
4	도시3	도시4	도시2
5	도시4	도시2	도시3
6	도시4	도시3	도시2

TSP (Traveling Salesperson Problem) 예제

- 시작 및 종료 도시를 '도시 1'로 고정했을 때,
각 경로에 대한 비용 계산

	도시 1	도시 2	도시 3	도시 4
도시 1	0	10	15	20
도시 2	10	0	35	25
도시 3	15	35	0	30
도시 4	20	25	30	0

경로	1	2	3	총비용
1	도시2	도시3	도시4	
2	도시2	도시4	도시3	
3	도시3	도시2	도시4	
4	도시3	도시4	도시2	
5	도시4	도시2	도시3	
6	도시4	도시3	도시2	

TSP (Traveling Salesperson Problem) 예제

- 시작 및 종료 도시를 '도시 1' 로 고정 했을 때,
각 경로에 대한 비용 계산
- 경로1: 도시1 → 2 → 3 → 4 → 1**

	도시 1	도시 2	도시 3	도시 4
도시 1	0	10	15	20
도시 2	10	0	35	25
도시 3	15	35	0	30
도시 4	20	25	30	0

경로	1	2	3	총비용
1	도시2	도시3	도시4	
2	도시2	도시4	도시3	
3	도시3	도시2	도시4	
4	도시3	도시4	도시2	
5	도시4	도시2	도시3	
6	도시4	도시3	도시2	

TSP (Traveling Salesperson Problem) 예제

- 시작 및 종료 도시를 '도시 1' 로 고정 했을 때,
각 경로에 대한 비용 계산
- 경로1: 도시1 → 2 → 3 → 4 → 1**
- 비용: $10 + 35 + 30 + 20 = 95$**

	도시 1	도시 2	도시 3	도시 4
도시 1	0	10	15	20
도시 2	10	0	35	25
도시 3	15	35	0	30
도시 4	20	25	30	0

경로	1	2	3	총비용
1	도시2	도시3	도시4	95
2	도시2	도시4	도시3	
3	도시3	도시2	도시4	
4	도시3	도시4	도시2	
5	도시4	도시2	도시3	
6	도시4	도시3	도시2	

TSP (Traveling Salesperson Problem) 예제

- 시작 및 종료 도시를 '도시 1'로 고정했을 때,
각 경로에 대한 비용 계산
- 나머지 경로에 대해서도 모두 계산**

	도시 1	도시 2	도시 3	도시 4
도시 1	0	10	15	20
도시 2	10	0	35	25
도시 3	15	35	0	30
도시 4	20	25	30	0

경로	1	2	3	총비용
1	도시2	도시3	도시4	95
2	도시2	도시4	도시3	...
3	도시3	도시2	도시4	...
4	도시3	도시4	도시2	...
5	도시4	도시2	도시3	...
6	도시4	도시3	도시2	...

TSP (Traveling Salesperson Problem) 예제

- 모든 도시(4개 도시)에 대하여, 각각 시작 및 종료 도시로 고정 했을 때,
가능한 모든 경로의 수
 - 시작 종료 도시를 선택하는 경우의 수: 4개
 - 나머지 3개 도시를 순서대로 선택하는 경우의 수: 6개
- ➔ 4개 도시에 대한 모든 경로의 수: 24개
- ➔ n 개 도시에 대한 전체 경로의 수: $n!$ 개

대표적인 NP-Complete 문제: 0/1 Knapsack (배낭 문제)



0/1 Knapsack (배낭 문제)

- 무게 제한이 있는 배낭에 가치가 최대가 되도록 물건들을 골라 담는 문제
- 물건을 쪼갤 수 있다면(Greedy) 쉽게 풀리지만, 통째로 넣거나 빼야 하는 조건이 문제를 어렵게 만듦
- 모든 물건의 조합을 고려해야 하므로, 물건의 개수가 n 개일 때 조합은 2^n 으로 증가
- 동적계획법(DP)으로 풀 수 있는 경우도 있지만, 배낭의 무게가 매우 크면 여전히 Intractable 한 상황이 발생 할 수 있음



0/1 Knapsack (배낭 문제)

0/1 Knapsack (배낭 문제) 예시

- 무게 제한이 있는 배낭에 가치가 최대가 되도록 물건들을 골라 담으세요
 - 배낭 한도: 10kg
 - 물건의 무게와 가치
 - 물건 1: 6kg, 10만원
 - 물건 2: 5kg, 7만원
 - 물건 3: 4kg, 6만원
- 제약사항: 물건은 쪼갤 수 없음 (0/1 은 쪼갤 수 없으므로, 물건을 넣을지(1) 말지(0) 결정해야 하는 문제라는 것을 의미)



0/1 Knapsack (배낭 문제) 예시

- 물건마다 넣을지(1) 말지(0) 선택할 수 있음
 - 배낭에 물건을 넣는 모든 경우의 수
- ➔ 물건마다 0/1을 선택하는 모든 경우의 수: 8개

	물건1	물건2	물건3
1	0	0	0
2	0	0	1
3	0	1	0
4	0	1	1
5	1	0	0
6	1	0	1
7	1	1	0
8	1	1	1



0/1 Knapsack (배낭 문제)

0/1 Knapsack (배낭 문제) 예시

- 배낭의 무게(물건의 총 무게)를 계산
 - 배낭 한도: 10kg
 - 물건의 무게와 가치
 - 물건 1: **6kg**, 10만원
 - 물건 2: **5kg**, 7만원
 - 물건 3: **4kg**, 6만원

	물건1	물건2	물건3	무게
1	0	0	0	0
2	0	0	1	4
3	0	1	0	5
4	0	1	1	9
5	1	0	0	6
6	1	0	1	10
7	1	1	0	11
8	1	1	1	15



0/1 Knapsack (배낭 문제)

0/1 Knapsack (배낭 문제) 예시

- 배낭의 가치(물건의 총 가치)를 계산
- 배낭 한도: 10kg
- 물건의 무게와 가치
 - 물건 1: 6kg, **10만원**
 - 물건 2: 5kg, **7만원**
 - 물건 3: 4kg, **6만원**

	물건1	물건2	물건3	무게	가치
1	0	0	0	0	0
2	0	0	1	4	5
3	0	1	0	5	6
4	0	1	1	9	11
5	1	0	0	6	10
6	1	0	1	10	16
7	1	1	0	11	17
8	1	1	1	15	23



0/1 Knapsack (배낭 문제)

0/1 Knapsack (배낭 문제) 예시

- 제약조건을 고려한 최대가치 선택
- **배낭 한도: 10kg**
- 물건의 무게와 가치
 - 물건 1: 6kg, 10만원
 - 물건 2: 5kg, 7만원
 - 물건 3: 4kg, 5만원

	물건1	물건2	물건3	무게	가치
1	0	0	0	0	0
2	0	0	1	4	5
3	0	1	0	5	6
4	0	1	1	9	11
5	1	0	0	6	10
6	1	0	1	10	16
7	1	1	0	11	17
8	1	1	1	15	23



0/1 Knapsack (배낭 문제)

0/1 Knapsack (배낭 문제) 예시

- 최대가치: 16
 - 물건1 과 물건 3을 선택하는 경우
- 가능한 모든 조합의 개수는
 - ➔ 변수가 3개라면, 2^3
 - ➔ 변수가 N개라면, 2^n

	물건1	물건2	물건3	무게	가치
1	0	0	0	0	0
2	0	0	1	4	5
3	0	1	0	5	6
4	0	1	1	9	11
5	1	0	0	6	10
6	1	0	1	10	16
7	1	1	0	11	17
8	1	1	1	15	23

NP-Complete 문제 대응



NP-Complete 문제는 어떻게 처리할까...?

- 개발자가 직면한 문제가 NP-Complete임을 모르면, 영원히 나오지 않을 '완벽한 정답'을 찾으려 시간을 허비할 수 있습니다
- 무식하게(Brute-force) 모든 경우를 계산하려는 시도하는 것은 비효율적입니다
 - 예) 100%의 정답을 위해 100년동안 계산할 것인가?
 - 예) 0.0001% 더 좋은 경로를 찾기 위해 리소스에 수조 원을 쓸 것인가?



NP-Complete 문제는 어떻게 처리할까...?

- 이 문제가 NP-Complete라는 것을 알게 되는 순간, 전략을 '최적해'에서 **'충분히 좋은 해'**로 수정해야 합니다
- ➔ NP-Complete 문제들은 적절한 시간 내에 95~99%의 정답을 찾는 것이 목표